

SmartLock

Présentation du projet :

- *Le projet SmartLock modernise la gestion du matériel du DeVinci Fablab en remplaçant les serrures traditionnelles par un système d'accès sécurisé et connecté via badge NFC.*
- *Cette solution couple la sécurisation physique des armoires à une gestion centralisée des stocks pour garantir une traçabilité complète des emprunts en temps réel.*
- *A terme, la solution sera implémentée toutes les armoires du Fablab afin de sécuriser le matériel de l'espace.*

V2 - VF	Plan de vérification système	17/05/2026
Rédacteur : Florian Cazac	Relecteurs : Augustin Winterbert, Alexandre Houdin, Guilhem Henoux, Lois Halmaert, Yann Videmment	Jalon revue des tests
	Validateurs : Florian Cazac	





1. Introduction

Ce document décrit le plan de vérification système du projet SmartLock. Il couvre la recette de chaque sous-système, la matrice de tests système, les scénarios d'intégration bout-en-bout et les procédures de test pas-à-pas à suivre pour valider le fonctionnement complet du système.

1.1. Périmètre

Le plan couvre les 6 sous-systèmes suivants et leur intégration complète :

1.2. Légende des statuts

Symbole	Signification
Validé	Test exécuté avec succès, critères de recette atteints
En cours	Test en cours d'exécution ou sous-système partiellement validé
Bloqué	Test impossible en l'état - prérequis non satisfaits
Non commencé	Test planifié mais pas encore exécuté



2. Critères de recette par sous-système

2.1. PCB custom - Assemblage et soudure

Responsable : Guilhem Henoux

Statut : Validé

Aucun court-circuit entre les pistes (contrôle au multimètre)
Continuité vérifiée sur toutes les pistes critiques (alimentation, signaux I²C, GPIO)
Composants CMS 0402/0603/0805 correctement soudés (pas de faux contacts, pas de ponts)
Câble correctif optocoupler (broche 12V → broche 4 U5) en place et fonctionnel
Inspection visuelle : aucun composant mal orienté ou manquant

2.2. Régulateurs DC-DC (MC34063ADR)

Responsable : Guilhem Henoux

Statut : En cours - bloqué par les résistances R_SC (commande Mouser/LCSC en attente)

Tension de sortie U1 (12V → 5V) : $\geq 4.9V$ sans charge, $\geq 4.8V$ sous charge Raspberry Pi
Tension de sortie U2 (11.1V → 5V) : $\geq 4.9V$ sans charge, $\geq 4.8V$ sous charge écran 7"
Tension de sortie U3 (11.1V → 12V) : $\geq 11.8V$ pour alimentation solénoïde
Tension de sortie U4 (12V → 12.6V) : charge batterie à $\leq 250mA$ sans surtension
Démarrage complet du Raspberry Pi Zero 2W sans redémarrage intempestif
Démarrage de l'écran 7" HDMI sans coupure d'alimentation

2.3. BMS 3S et pack batteries Li-Ion

Responsable : Guilhem Henoux

Statut : En cours - intégration PCB en attente de la correction des régulateurs

Tension de chaque cellule $< 4.2V$ après charge complète (protection surcharge active)
Protection sous-tension active (coupure si cellule $< 2.5V$)
Protection court-circuit active (coupure instantanée)
Charge correcte via régulateur U4 : courant de charge $\sim 250mA$, tension finale $\sim 12.6V$
Basculement automatique sur batterie lors d'une coupure secteur (sans interruption système)
Autonomie mesurée ≥ 30 min sous charge nominale (Raspberry Pi + écran)



2.4. Intégration mécanique

Responsable : Alexandre Houdin

Statut : En cours - modèle CAO en cours d'ajustement

Tous les composants (PCB, écran, RPi, PN532, solénoïde, capteur Hall) positionnés dans le boîtier

Câblage sans contrainte mécanique sur les connecteurs

Boîtier fermable sans forcer (aucun conflit géométrique)

Solénoïde correctement aligné avec le mécanisme de verrouillage du casier

Capteur à effet Hall positionné pour détecter l'état ouvert/fermé du casier

Accès lecteur NFC PN532 visible et accessible depuis l'extérieur du boîtier

2.5. Interface Homme-Machine (IHM)

Responsable : Loïs Halmaert

Statut : Validé

Démarrage automatique de l'IHM au boot du Raspberry Pi (sans intervention manuelle)

Lecture NFC détectée en < 2 secondes après présentation de la carte

Toutes les vues accessibles et fonctionnelles (home, navigation, sélection, panier, validation, accès, clôture)

Timer d'inactivité actif : 30s sur home -> fermeture, 90s sur autres vues -> retour home

Timer d'alerte : 10 min de casier ouvert -> alerte Discord envoyée

Mode SIMULATION_MODE bascule sans erreur entre mode réel et simulé

Connexion à l'API confirmée (authentification Keycloak client_credentials réussie)

2.6. API d'authentification et d'autorisation

Responsable : Florian Cazac

Statut : Validé

87/87 tests automatisés (pytest) passants sans échec

Endpoint POST /auth/locker/{id}/check répond en < 500ms en conditions nominales

Authentification Keycloak opérationnelle (JWT valide, JWKS accessible)

Service accounts Raspberry Pi (smartlock-lockers) et NFC scanner (nfc-scanner) fonctionnels

Journal d'audit enregistre chaque tentative d'accès (accordée et refusée)



Base de données PostgreSQL accessible et migrations appliquées (alembic upgrade head)

API déployée via Docker Compose et accessible sur le réseau interne



3. Matrice de tests système

Le tableau ci-dessous liste l'ensemble des tests à réaliser au niveau système. Chaque test est identifié par un code SYS-XX, associé à un ou plusieurs sous-systèmes, à un responsable et à ses prérequis.

ID	Test	Sous-système(s)	Responsable	Prérequis	Statut	Observations
SYS-01	Démarrage complet sous alimentation secteur (RPi + écran)	PCB, DC-DC	Guilhem	R_SC remplacées	Bloqué	En attente composants
SYS-02	Démarrage complet sur batterie de secours	PCB, BMS, DC-DC	Guilhem	SYS-01	Bloqué	En attente SYS-01
SYS-03	Affichage IHM sur écran 7" Waveshare	DC-DC, RPi, IHM	Guilhem, Loïs	SYS-01	Bloqué	En attente SYS-01
SYS-04	Lecture NFC et identification utilisateur via API	IHM, RPi, API	Loïs, Florian	SYS-03	Validé	Testé en SIMULATION_MODE
SYS-05	Ouverture casier sur badgeage autorisé (solénoïde)	IHM, API, PCB	Florian, Loïs, Guilhem	SYS-04	Validé	
SYS-06	Refus d'accès - carte NFC inconnue	IHM, API	Florian	SYS-04	Validé	
SYS-07	Refus d'accès - compte utilisateur révoqué	IHM, API	Florian	SYS-04	Validé	
SYS-08	Refus d'accès - absence de permission sur le casier	IHM, API	Florian	SYS-04	Validé	
SYS-09	Refus d'accès - permission expirée (valid_until dépassé)	IHM, API	Florian	SYS-04	Validé	
SYS-10	Détection fermeture casier (capteur à effet Hall)	PCB, RPi, IHM	Guilhem, Loïs	SYS-05	Non commencé	



SYS-11	Journal d'audit enregistré après chaque accès	API	Florian	SYS-05	Validé	Vérifié via GET /logs
SYS-12	Comportement dégradé lors d'une coupure API	IHM, RPi	Loïs	SYS-03	Non commencé	Attendre timeout, afficher message erreur
SYS-13	Alerte Discord après 10 min de casier ouvert	IHM	Loïs	SYS-05	Validé	Timer_manager vérifié
SYS-14	Basculement automatique sur batterie (coupure secteur simulée)	PCB, BMS	Guilhem	SYS-02	Bloqué	En attente intégration BMS



4. Procédures de test pas-à-pas

4.1. Prerequisites techniques

Variables a definir avant de commencer (adapter selon votre .env) :

```
API_URL="https://api.smartlock.devinci-fablab.fr"
KC_URL="https://auth.smartlock.devinci-fablab.fr"
REALM="smartlock"           # valeur KEYCLOAK_REALM dans .env
LOCKER_ID=1                 # ID du casier cible
CARD_UID="04:AB:CD:EF:12:34" # UID brut de la carte NFC (lu par le PN532)
```

Obtenir le token RPi (client smartlock-lockers, grant client_credentials) :

```
RPI_TOKEN=$(curl -s -X POST \
  "$KC_URL/realms/$REALM/protocol/openid-connect/token" \
  -d "grant_type=client_credentials" \
  -d "client_id=smartlock-lockers" \
  -d "client_secret=$LOCKER_CLIENT_SECRET" \
  | jq -r '.access_token')
```

Obtenir un token admin (grant password) :

```
ADMIN_TOKEN=$(curl -s -X POST \
  "$KC_URL/realms/$REALM/protocol/openid-connect/token" \
  -d "grant_type=password" \
  -d "client_id=smartlock-api" \
  -d "username=$ADMIN_USER" \
  -d "password=$ADMIN_PASSWORD" \
  | jq -r '.access_token')
```

Outils requis : curl, jq (Windows : winget install jqlang.jq)

4.2. Prerequisites techniques

Variables a definir avant de commencer (adapter selon votre .env) :

```
API_URL="https://api.smartlock.devinci-fablab.fr"
KC_URL="https://auth.smartlock.devinci-fablab.fr"
REALM="smartlock"           # valeur KEYCLOAK_REALM dans .env
LOCKER_ID=1                 # ID du casier cible
CARD_UID="04:AB:CD:EF:12:34" # UID brut de la carte NFC (lu par le PN532)
```

Obtenir le token RPi (client smartlock-lockers, grant client_credentials) :

```
RPI_TOKEN=$(curl -s -X POST \
  "$KC_URL/realms/$REALM/protocol/openid-connect/token" \
  -d "grant_type=client_credentials" \
  -d "client_id=smartlock-lockers" \
  -d "client_secret=$LOCKER_CLIENT_SECRET" \
  | jq -r '.access_token')
```

Obtenir un token admin (grant password) :

```
ADMIN_TOKEN=$(curl -s -X POST \
  "$KC_URL/realms/$REALM/protocol/openid-connect/token" \
  -d "grant_type=password" \
  -d "client_id=smartlock-api" \
  -d "username=$ADMIN_USER" \
  -d "password=$ADMIN_PASSWORD" \
  | jq -r '.access_token')
```

Outils requis : curl, jq (Windows : winget install jqlang.jq)



4.3. Procédure A : Test d'accès complet - scénario nominal

Matériel nécessaire : alimentation secteur 220V, carte NFC enregistrée, multimètre, PC avec accès à l'API.

Étape 1 - Mise sous tension :

Connecter l'alimentation secteur 220V au connecteur J8 du PCB
Vérifier la LED verte d'alimentation sur le PCB (confirme la tension 5V disponible)
Mesurer la tension au multimètre sur le connecteur Raspberry Pi : doit être entre 4.8V et 5.2V

Étape 2 - Démarrage système :

Attendre le démarrage complet du Raspberry Pi Zero 2W (environ 45 secondes)
L'écran 7" Waveshare doit afficher l'IHM SmartLock dans les 60 secondes
Si l'écran reste noir après 90s, vérifier la tension U2 (problème régulateur)

Étape 3 - Vérification réseau et API :

Vérifier que le Raspberry Pi est connecté au réseau (LED réseau active ou ping depuis PC)
Tester l'accessibilité de l'API : GET <https://api.smartlock.devinci-fablab.fr/health>
Réponse attendue : HTTP 200

Étape 4 - Test NFC et accès casier :

Prendre une carte NFC enregistrée (utilisateur avec permission can_open sur le casier)
Présenter la carte devant le lecteur PN532 à moins de 5 cm

```
# Simulation depuis un terminal (sans le lecteur physique)
curl -s -X POST "$API_URL/auth/locker/$LOCKER_ID/check" \
  -H "Authorization: Bearer $RPI_TOKEN" \
  -H "Content-Type: application/json" \
  -d "{\"card_id\": \"$CARD_UID\"}"
# Reponse attendue :
# {"allowed": true, "display_name": "Prenom Nom", "permissions": {"permission_level":
"can_open"}}
# Simulation depuis un terminal (sans le lecteur physique)
curl -s -X POST "$API_URL/auth/locker/$LOCKER_ID/check" \
  -H "Authorization: Bearer $RPI_TOKEN" \
  -H "Content-Type: application/json" \
  -d "{\"card_id\": \"$CARD_UID\"}"
# Reponse attendue :
# {"allowed": true, "display_name": "Prenom Nom", "permissions": {"permission_level":
"can_open"}}
```

Observer l'IHM : écran de bienvenue avec nom de l'utilisateur dans les 2 secondes
Observer le déclenchement du relais GPIO (clic audible ou LED indicatrice)
Vérifier l'ouverture mécanique du casier



Étape 5 – Vérification du journal d'audit :

Depuis un PC, appeler GET /logs (avec token admin)

```
curl -s -H "Authorization: Bearer $ADMIN_TOKEN" \
"$API_URL/logs/?locker_id=$LOCKER_ID" | jq '.[0]'
# Reponse attendue :
# { "result": "allowed", "can_open": true, "user_id": "...", "username": "..." }
curl -s -H "Authorization: Bearer $ADMIN_TOKEN" \
"$API_URL/logs/?locker_id=$LOCKER_ID" | jq '.[0]'
# Reponse attendue :
# { "result": "allowed", "can_open": true, "user_id": "...", "username": "..." }
```

Vérifier la présence d'une entrée : allowed=true, user_id correct, timestamp récent

Étape 6 – Fermeture et retour à l'accueil :

Fermer physiquement le casier

Vérifier que le capteur Hall détecte la fermeture (LED ou log)

L'IHM doit revenir à l'écran d'accueil automatiquement

Critères de succès : Toutes les étapes ci-dessus réussies sans erreur.

4.4. Procédure B : Tests de refus d'accès (INT-2 à INT-5)

Utiliser le meme token RPi qu'en Procedure A. Changer uniquement CARD_UID pour chaque scenario.

```
# INT-2 : carte inconnue
curl -s -X POST "$API_URL/auth/locker/$LOCKER_ID/check" \
-H "Authorization: Bearer $RPi_TOKEN" \
-H "Content-Type: application/json" \
-d '{"card_id": "00:00:00:00:00"}'
# Attendu : {"allowed": false, "reason": "card_not_registered"}
# INT-3 : compte revoke
# 1. Revoquer le compte via API admin :
curl -s -X POST "$API_URL/users/$USER_ID/revoke" \
-H "Authorization: Bearer $ADMIN_TOKEN"
# 2. Scanner la meme carte qu'en INT-1
# Attendu : {"allowed": false, "reason": "account_revoked"}
# INT-4 : pas de permission sur ce casier
# Utiliser une carte d'un utilisateur sans entree dans locker_permission pour ce casier
# Attendu : {"allowed": false, "reason": "no_permission"}
# INT-5 : permission expiree
# Utiliser une carte d'un utilisateur dont valid_until est dans le passe
# (modifier la permission via PUT /lockers/{id}/permissions/{perm_id} avec valid_until
passe)
# Attendu : {"allowed": false, "reason": "no_permission"}
```



Utiliser le meme token RPi qu'en Procedure A. Changer uniquement CARD_UID pour chaque scenario.

```
# INT-2 : carte inconnue
curl -s -X POST "$API_URL/auth/locker/$LOCKER_ID/check" \
  -H "Authorization: Bearer $RPi_TOKEN" \
  -H "Content-Type: application/json" \
  -d '{"card_id": "00:00:00:00:00"}'
# Attendu : {"allowed": false, "reason": "card_not_registered"}

# INT-3 : compte revoke
# 1. Revoquer le compte via API admin :
curl -s -X POST "$API_URL/users/$USER_ID/revoke" \
  -H "Authorization: Bearer $ADMIN_TOKEN"
# 2. Scanner la meme carte qu'en INT-1
# Attendu : {"allowed": false, "reason": "account_revoked"}

# INT-4 : pas de permission sur ce casier
# Utiliser une carte d'un utilisateur sans entree dans locker_permission pour ce casier
# Attendu : {"allowed": false, "reason": "no_permission"}

# INT-5 : permission expiree
# Utiliser une carte d'un utilisateur dont valid_until est dans le passe
# (modifier la permission via PUT /lockers/{id}/permissions/{perm_id} avec valid_until
passe)
# Attendu : {"allowed": false, "reason": "no_permission"}
```

Exécuter les 4 scénarios de refus successivement. Chaque scénario doit produire un refus et un log correspondant.

B1 - Carte inconnue :

Utiliser une carte NFC vierge (non enregistrée dans Keycloak)
Présenter au lecteur PN532 - résultat attendu : refus, log card_not_registered

B2 - Compte révoqué :

Via l'API admin : POST /users/{id}/revoke pour désactiver un compte test
Présenter la carte de cet utilisateur - résultat attendu : refus, log account_revoked
Restaurer le compte après le test : POST /users/{id}/restore

B3 - Absence de permission :

Utiliser une carte d'un utilisateur actif sans permission sur le casier cible
Présenter au lecteur - résultat attendu : refus, log no_permission

B4 - Permission expirée :

Configurer une permission avec valid_until = date dans le passé (ex. 2026-01-01)
Présenter la carte de cet utilisateur - résultat attendu : refus, log expired
Supprimer la permission test après vérification

Vérification finale : Consulter GET /logs pour confirmer les 4 entrées denied avec les bons motifs.



4.5. Procédure C : Test de robustesse – 25 badgeages consécutifs

```
# Boucle 25 badgeages consécutifs (Linux/macOS/WSL)
for i in $(seq 1 25); do
  RESULT=$(curl -s -X POST "$API_URL/auth/locker/$LOCKER_ID/check" \
    -H "Authorization: Bearer $RPI_TOKEN" \
    -H "Content-Type: application/json" \
    -d '{"card_id": \"$CARD_UID\"}')
  echo "Scan $i : $(echo $RESULT | jq -r '.allowed')"
  sleep 0.5
done

# Verifier que 25 logs ont ete crees :
curl -s -H "Authorization: Bearer $ADMIN_TOKEN" \
  "$API_URL/logs/?locker_id=$LOCKER_ID" | jq 'length'
# Attendu : 25

# Boucle 100 badgeages consécutifs (Linux/macOS/WSL)
for i in $(seq 1 25); do
  RESULT=$(curl -s -X POST "$API_URL/auth/locker/$LOCKER_ID/check" \
    -H "Authorization: Bearer $RPI_TOKEN" \
    -H "Content-Type: application/json" \
    -d '{"card_id": \"$CARD_UID\"}')
  echo "Scan $i : $(echo $RESULT | jq -r '.allowed')"
  sleep 0.5
done

# Verifier que 25 logs ont ete crees :
curl -s -H "Authorization: Bearer $ADMIN_TOKEN" \
  "$API_URL/logs/?locker_id=$LOCKER_ID" | jq 'length'
# Attendu : 25
```

Ce test vérifie la stabilité du système sous sollicitation répétée. À réaliser une fois tous les sous-systèmes validés.

Préparation :

Système complet sous tension depuis au moins 10 minutes (température stabilisée)

Un utilisateur avec permission active sur le casier, carte NFC prête

Script de monitoring actif (logs API en temps réel : docker compose logs -f api)

Exécution :

Effectuer 25 badgeages consécutifs en respectant un délai de 10 secondes entre chaque

Après chaque badgeage, fermer le casier avant le suivant

Noter tout comportement anormal (lenteur, erreur IHM, refus inattendu, déconnexion)

Vérification :

Consulter GET /logs : 25 entrées allowed=true attendues

Aucune erreur dans les logs Docker (docker compose logs api | grep ERROR)

Température du PCB mesurée au multimètre infra-rouge : doit rester < 60°C

Tension alimentation mesurée après test : doit être identique à la mesure initiale (pas de dérive)



Critères de succès : 25/25 badgeages réussis, aucune erreur, températures et tensions stables.